A PROJECT REPORT
ON
**FOUNDATION COURSE**

# SMARTMED
## MEDICATION MANAGEMENT
## &
## ADHERENCE MONITORING SYSTEM

**Submitted to:**

Rajasthan Technical University, Kota (Raj.)

**Submitted By:**


**Aditya Mishra (24EARCS010) (CSE-A)**

**Aditya Yadav (24EARCS011) (CSE-A)**

**Badal Verma (24EARCS038) (CSE-A)**

**Bhawna Sen (24EARCS039) (CSE-A)**


**Session: 2026-27**

---


Department of Computer Science & Engineering
**ARYA COLLEGE OF ENGINEERING & I.T.**
**SP-42, RIICO INDUSTRIAL AREA, KUKAS, JAIPUR**

ARYA College of Engineering & Information Technology
(Approved by AICTE & Affiliated to RTU, Kota)
SP-42, RIICO Industrial Area, Delhi Road, Kukas, Jaipur - 302028 (Rajasthan) India
Tel. No. : 0141-6604555
Toll Free : 1800-266-2000
Email Id : info@aryacollege.in
Website : www.aryacollege.in
Estd. Yr. 2000
ACCREDITED BY: NAAC NATIONAL ASSESSMENT AND ACCREDITATION COUNCIL | NBA NATIONAL BOARD OF ACCREDITATION FOR ECE & IT

## Department of Computer Science & Engineering

# CERTIFICATE OF APPROVAL

The Project Report entitled "**SMARTMED MEDICATION MANAGEMENT & ADHERENCE MONITORING SYSTEM**" submitted by "Aditya Mishra (24EARCS010)**,** Aditya Yadav (24EARCS011), Badal Verma (24EARCS038), Bhawna Sen (24EARCS039)**"** has been examined by us and is hereby approved for carrying out the project leading to the award of degree "**Bachelor of Technology in Computer Science & Engineering"**. By this approval the undersigned does not necessarily endorse or approve any statement made, opinion expressed or conclusion drawn therein, but approve the pursuance of project only for the above mentioned purpose.

**Mentor**
**Mohit Mishra**
Professor

## Department of Computer Science & Engineering

### CERTIFICATE

This is to certify that the work embodies in this Project entitled "**SMARTMED MEDICATION MANAGEMENT & ADHERENCE MONITORING SYSTEM**" being submitted by "Aditya Mishra (24EARCS010), Aditya Yadav (24EARCS011), Badal Verma (24EARCS038), Bhawna Sen (24EARCS039)" in partial fulfillment of the requirement for the award of "**Bachelor of Technology in Computer Science & Engineering**" to Rajasthan Technical University, Kota (Raj.) during the academic year 2026-27 is a record of bonafide piece of work, carried out by them under our supervision and guidance in the **"Department of Computer Science & Engineering", Arya College of Engineering & Information Technology, Jaipur**.

**Mentor**
**Mohit Mishra**
**Professor**

**HOD**
**Dr.Akhil Pandey**
**Professor**

## Department of Computer Science & Engineering

## DECLARTION

We "**Aditya Mishra (24EARCS010), Aditya Yadav (24EARCS011), Badal Verma (24EARCS038), Bhawna Sen (24EARCS039)**", students of **Bachelor of Technology in Computer Science & Engineering , session 2026-27, Arya College of Engineering & Information Technology, Jaipur**, here by declare that the work presented in this Project entitled "**SMARTMED MEDICATION MANAGEMENT & ADHERENCE MONITORING SYSTEM**" is the outcome of our own work, is bonafide and correct to the best of our knowledge and this work has been carried out taking care of Engineering Ethics. The work presented does not infringe any patented work and has not been submitted to any other University or anywhere else for the award of any degree or any professional diploma.

**Aditya Mishra (24EARCS010)**
**Aditya Yadav (24EARCS011)**
**Badal Verma (24EARCS038)**
**Bhawna Sen (24EARCS039)**

# Abstract

SmartMed is a web-based medication management and adherence monitoring application designed to help patients organize medicines, schedules and dose records while supporting consent-based monitoring by caregivers and doctors. The project is submitted by Team Alpha Beta Crew from Arya College of Engineering & IT, Kukus, under the supervision of Prof. Ram Babu Sir.

The system combines a React and TypeScript frontend with a Java 17 and Spring Boot backend and a MySQL relational database. The application provides account registration and login, medication management, medication schedules, dose tracking, adherence history and analytics, care-team relationships, a medication interaction-checking interface and an in-app notification centre. Authentication is based on JSON Web Tokens (JWT), while password credentials are stored using BCrypt hashing. Patients control caregiver and doctor access through explicit relationship invitations and revocation.

The backend is organized into controller, service, repository, entity, DTO, security and configuration layers. Flyway migrations manage database schema evolution, and integration tests cover major functional and security behaviours. The interaction-checking module uses a provider abstraction; the current default mock provider does not contain real clinical interaction data. SmartMed is an educational software project and is not a medical device. It does not diagnose, prescribe or replace advice from qualified healthcare professionals. Keywords Medication management, adherence monitoring, React, TypeScript, Spring Boot, Java, MySQL, JWT, caregiver monitoring, notifications.

# TABLE OF CONTENTS


| Section | Page |
|---|---|
| Certificate of Approval | 2 |
| Certificate | 3 |
| Declaration | 4 |
| Acknowledgement | 5 |
| Abstract | 6 |
| Chapter 1: Introduction | 8 |
| Chapter 2: Existing System and Proposed System | 10 |
| Chapter 3: Feasibility Study and Requirements | 12 |
| Chapter 4: System Analysis and Design | 14 |
| Chapter 5: Technology Stack and Architecture | 16 |
| Chapter 6: Database Design | 18 |
| Chapter 7: Implementation and Functional Modules | 20 |
| Chapter 8: User Workflows | 23 |
| Chapter 9: Testing and Results | 25 |
| Chapter 10: Conclusion and Future Scope | 27 |
| References | 28 |
| Appendix A: Local Setup Summary | 29 |
| Appendix B: Important Safety and Security Notes | 30 |


.

# CHAPTER 1: INTRODUCTION

## 1.1 Overview

Medication adherence is an important part of day-to-day healthcare. People may need to take multiple medicines at different times, maintain a history of doses and communicate progress to a caregiver or doctor. Paper notes, alarms and disconnected applications can make this information difficult to organize consistently.

SmartMed provides a centralized digital workspace for medication records, schedules, dose status and adherence summaries. It is built as a browser-based application with a frontend that communicates with a versioned REST API. The backend validates requests, authenticates users and applies ownership and role rules before reading or modifying data.

## 1.2 Problem Statement

The project addresses the lack of a single, structured place to maintain medication schedules and review recorded adherence. It also addresses the need to share selected monitoring information with trusted caregivers or doctors without giving them unrestricted control over a patient's records.

## 1.3 Objectives

- Provide secure patient registration and login.
- Allow patients to maintain medication information and schedules.
- Record scheduled doses and their status, and retain historical records.
- Present adherence history and analytics over a selected date range.
- Support patient-controlled caregiver and doctor relationships.
- Provide an informational interaction-checking workflow and an in-app notification centre.
- Use a layered architecture and automated tests to support maintainability.

## 1.4 Scope

SmartMed is intended for educational demonstration and personal organization. It does not diagnose conditions, prescribe medicines, issue emergency alerts or guarantee that a medicine combination is safe. Interaction results depend on the configured provider, which is a mock implementation in the submitted project.

## 1.5 Need and Significance

A structured application can make medication information easier to review and reduce reliance on scattered notes. A historical record also makes it possible to calculate simple adherence measures from saved dose states. The consent-based monitoring model demonstrates a privacy-conscious way to share selected information with a care team.

### 1.6 Limitations

- The system depends on correct data entered by the user.
- The current interaction provider is a mock provider and supplies no real clinical interaction database.
- Analytics are based on recorded application events and should not be interpreted as clinical measurements.
- Care-team monitoring is read-only and requires an active relationship.
- The application requires a correctly configured database and backend service.

### 1.7 Organization of Report

Chapter 2 reviews the existing approach and the proposed solution. Chapter 3 presents feasibility and requirements. Chapter 4 explains analysis and design. Chapter 5 describes architecture and technologies. Chapter 6 discusses data design. Chapter 7 explains functional modules. Chapter 8 describes workflows and access control. Chapter 9 outlines testing. Chapter 10 presents conclusions and future enhancements.

# CHAPTER 2: EXISTING SYSTEM AND PROPOSED SYSTEM

## 2.1 Existing System

Medication routines are often managed through paper prescriptions, personal calendars, alarms or separate health applications. Such approaches can be useful, but information may become fragmented. Paper records are difficult to search and summarize automatically, while isolated reminders may not provide a complete history or a controlled way to share selected information with a caregiver.

## 2.2 Drawbacks of Existing Approach

- Medication lists and dose records may be stored in different places.
- Historical adherence may require manual review and calculation.
- Sharing records with caregivers can be inconsistent or overly broad.
- Changes to schedules may not be reflected in older notes.
- Manual tracking makes long-term summaries more difficult.

## 2.3 Proposed System

SmartMed brings medication details, schedules, dose records and adherence analytics into one web application. A patient can invite a caregiver or doctor and later revoke that access. The server checks the authenticated account and active relationship before returning monitoring data.

## 2.4 Advantages

- Centralized medication and schedule management.
- Date-filtered dose history and adherence summaries.
- Role-based and ownership-aware API access.
- Patient-controlled care relationships.
- Notification centre for selected application events.
- Layered implementation with database migrations and automated tests.

## 2.5 Comparative Study

| Feature | Manual / Disconnected Tools | SmartMed |
|---|---|---|
| Medication records | Separate notes or applications | Centralized records per account |
| Schedule and dose tracking | Manual or reminder-only | Schedules and saved dose states |
| Adherence review | Manual calculation | Date-filtered history and analytics |
| Care-team sharing | Informal or broad sharing | Explicit invitation and revocation |
| Security controls | Varies by tool | JWT authentication, roles and ownership checks |
| Clinical interaction data | Depends on separate resource | Provider interface; current provider is mock |

### 2.6 Feasibility of the Proposed Approach

The proposed system is technically feasible because it uses established web technologies and a relational database. React supports a component-based user interface, Spring Boot provides REST APIs and dependency injection, and MySQL stores related records. The project can be run locally with a Java 17 runtime, Node.js/npm and a MySQL instance.

### 2.7 Summary

SmartMed improves organization and review of medication-related records while keeping clinical claims deliberately limited. Its strongest design feature is the separation of patient-owned operations from consent-based, read-only monitoring.

# CHAPTER 3: FEASIBILITY STUDY AND REQUIREMENTS

## 3.1 Technical Feasibility

The project uses widely adopted technologies and clear module boundaries. The backend uses Java 17 and Spring Boot 3, with Spring Web, Spring Data JPA, Spring Security, Bean Validation, Actuator and Flyway. The frontend uses React 18, Vite 5 and TypeScript. MySQL 8 provides persistent relational storage. Maven and npm support builds and tests.

## 3.2 Operational Feasibility

The system is designed around familiar tasks: create an account, add medication information, configure a schedule, review dose records and inspect summaries. Caregivers and doctors have separate monitoring views, subject to patient consent. Users should still confirm medication decisions with qualified professionals.

## 3.3 Economic Feasibility

The project can be developed using open-source frameworks and local development tools. Operational costs depend on hosting, database capacity, backups, domain configuration and monitoring. The source repository describes a deployment model using a frontend host, backend host and managed MySQL; actual deployment must be configured and verified separately.

## 3.4 Hardware and Software Requirements

| Category | Requirement |
|---|---|
| Processor | Modern dual-core or better |
| Memory | 8 GB RAM recommended for development |
| Operating System | Windows, Linux or macOS |
| Backend | JDK 17+, Maven wrapper |
| Frontend | Node.js 18+ and npm |
| Database | MySQL 8 or compatible configured instance |
| Browser | Modern browser with JavaScript enabled |
| Tools | IDE/editor, Git, terminal, API testing utility |

## 3.5 Functional Requirements

- FR-01: Register a patient account using name, email and password.
- FR-02: Authenticate users and return an access token.
- FR-03: Return the authenticated user's profile without password fields.
- FR-04: Create, view, update and deactivate medication records subject to ownership.
- FR-05: Maintain medication schedules and dose records.
- FR-06: View adherence history and analytics for date ranges.
- FR-07: Create, accept, decline and revoke care relationships.
- FR-08: Allow connected caregivers/doctors to read permitted monitoring data only.
- FR-09: Request an interaction check for medications owned by the authenticated patient.
- FR-10: List and update recipient-owned notifications.

### 3.6 Non-Functional Requirements

- Security: authentication, password hashing, ownership checks and role restrictions.
- Reliability: persistent records and controlled dose-state transitions.
- Maintainability: layered architecture and separated frontend/backend code.
- Usability: clear navigation and date-oriented summaries.
- Compatibility: modern browsers and standard REST/JSON communication.
- Configurability: secrets, database details, CORS and timezone provided through environment variables.

### 3.7 Assumptions and Constraints

Users provide accurate medication information and maintain their own account credentials. The backend must be configured with a secure JWT secret and database settings. The interaction-checking module is informational and is not connected to a production clinical interaction service.

# CHAPTER 4: SYSTEM ANALYSIS AND DESIGN

## 4.1 System Architecture Overview

The system follows a three-part runtime topology: React/TypeScript frontend, Spring Boot REST backend and MySQL database. The frontend sends HTTPS JSON requests to the API. JWT access tokens accompany authenticated requests. The backend applies request validation, authentication, role and ownership rules before using repositories to access data.

| Layer | Main Responsibility |
|---|---|
| Presentation | React pages, reusable components, dashboards, forms and date-based views |
| API client | Typed requests, token attachment, response/error handling |
| Controller | REST routes, input binding and response DTOs |
| Service | Business rules, transactions, access checks and workflows |
| Repository | Database reads, writes, projections and bounded queries |
| Persistence | MySQL tables managed by Flyway migrations |

## 4.2 Logical Architecture Diagram

| USER / BROWSER |
|---|
| React + TypeScript UI |
| REST / JSON + JWT |
| Spring Boot API |
| Controller → Service → Repository |
| MySQL 8 + Flyway schema |

Figure 4.1: Logical request path from the user interface through the API layers to persistent storage. Responses return through the same layers to the frontend.

## 4.3 Use Case Summary

| Actor | Use Cases |
|---|---|
| Patient | Register/login, manage medicines and schedules, record doses, review analytics, manage care consent, check interactions, read notifications |
| Caregiver | Accept/decline relationship, view connected patient summaries and permitted adherence information |
| Doctor | Accept/decline relationship, view connected patient overview and permitted adherence information |
| System | Validate tokens, enforce access rules, generate dose records, maintain notifications and apply database migrations |

## 4.4 Data Flow and Request Lifecycle

- The user performs an action in the React interface.
- The frontend API client serializes the request and attaches the access token when required.
- Spring Security validates the token and builds the authenticated principal.
- The controller validates the request and delegates to the service layer.
- The service checks account ownership, role and active care relationship where applicable.
- The repository reads or updates MySQL within a transaction.
- The backend returns a consistent success or error envelope to the client.

### 4.5 Security Design

The backend uses stateless JWT authentication. Passwords are stored as BCrypt hashes. The JWT subject contains the stable user identifier; the backend reloads the account and checks the token subject against the database identity. Authority decisions use the current database account rather than trusting a role value supplied by the browser. Public registration creates patient accounts only.

### 4.6 Relationship State Model

| Current State | Action | Next State |
|---|---|---|
| No relationship | Patient sends invitation | PENDING |
| PENDING | Invitee accepts | ACTIVE |
| PENDING | Invitee declines | REVOKED |
| ACTIVE | Patient revokes access | REVOKED |
| REVOKED | New invitation is created | New PENDING record |

### 4.7 Date and Time Handling

The backend uses an injected clock and a configurable IANA timezone, defaulting to Asia/Kolkata. Date-only values are kept as calendar dates instead of being unnecessarily converted through UTC instants. The frontend timezone should be configured consistently with the backend.

# CHAPTER 5: TECHNOLOGY STACK AND ARCHITECTURE

## 5.1 Frontend Technologies

**React 18:** builds the interface from reusable components. **TypeScript:** adds static typing to API data and component logic. **Vite 5:** provides local development and production builds. Chart.js and react-chartjs-2 are included for chart-based summaries. CSS modules and global style variables support component-specific and shared styling.

## 5.2 Backend Technologies

**Java 17:** runtime and language. **Spring Boot 3:** application framework. **Spring Web:** REST endpoints. **Spring Data JPA:** persistence and repository access. **Spring Security:** authentication and authorization. **Bean Validation:** request validation. **Actuator:** liveness/readiness endpoints. **Flyway:** ordered database migrations.

## 5.3 Database and Build Tools

MySQL 8 stores user, medication, schedule, dose, relationship and notification data. Flyway migrations are authoritative for schema evolution. Hibernate is configured to validate the schema rather than silently create or update production tables. Maven builds the backend; npm scripts build and test the frontend.

## 5.4 Technology Table

| Technology | Purpose |
|---|---|
| React 18 | Component-based browser interface |
| TypeScript | Typed frontend code |
| Vite 5 | Development server and frontend bundling |
| Java 17 / Spring Boot 3 | Backend application and REST API |
| Spring Security / JWT | Stateless authentication and request protection |
| Spring Data JPA | Object-relational persistence |
| MySQL 8 | Relational database |
| Flyway | Versioned schema migrations |
| JUnit / Spring Boot Test / H2 | Automated backend tests |

## 5.5 API DESIGN

The API is versioned under **/api/v1**. Successful responses use a common envelope containing a success flag, optional message and data. Error responses contain a failure flag and a structured error with a code and message. Authenticated routes require a Bearer access token.

| Method | Endpoint | Purpose |
|---|---|---|
| GET | /api/v1/health | Public process health response |
| POST | /api/v1/auth/register | Create a patient account |
| POST | /api/v1/auth/login | Authenticate an account |
| GET | /api/v1/users/me | Get current account profile |
| POST | /api/v1/interactions/check | Request informational interaction check |
| GET | /api/v1/notifications | List recipient notifications |
| GET | /api/v1/relationships | List relevant care relationships |

### 5.6 Example Response Structure

```
{
  "success": true,
  "message": null,
  "data": { }
}
```

The exact data object varies by endpoint. Error responses use a structured error code such as INVALID_CREDENTIALS. Clients should use the HTTP status and response body together rather than treating every response as successful.

### 5.7 Deployment Topology

The repository describes a deployment option with the frontend hosted on Vercel, the Spring Boot backend on Railway and MySQL on Railway. Production deployment requires explicit configuration of database connection settings, TLS mode, CORS origins, timezone and a strong JWT secret. The repository documentation is configuration guidance; it does not by itself prove that a production deployment has been tested.

# CHAPTER 6: DATABASE DESIGN

## 6.1 Overview

SmartMed uses a relational data model. The core domain entities include User, Medication, MedicationSchedule, DoseRecord, CareRelationship and Notification. Foreign keys and unique constraints support data integrity. Flyway migration files V1 through V4 define the schema and subsequent changes.

## 6.2 Core Entities

| Entity | Purpose | Representative Data |
|---|---|---|
| User | Account and role | ID, name, email, password hash, role |
| Medication | Patient-owned medication | ID, owner, name, strength/details, active state |
| MedicationSchedule | Dose timing definition | Medication, frequency, time, active state |
| DoseRecord | Scheduled dose occurrence | Schedule, date, time, status |
| CareRelationship | Consent between patient and care-team user | Patient, related user, relationship type, status |
| Notification | Recipient-owned event | Recipient, type, event key, read state, created time |

## 6.3 Entity Relationship Summary

A user can own multiple medication records. A medication can have one or more schedules, and a schedule can generate many dose records across calendar dates. A patient may have multiple care relationships with caregivers or doctors. Notifications belong to a recipient and can refer to a related application event.

| Relationship | Cardinality |
|---|---|
| User → Medication | One-to-many |
| Medication → MedicationSchedule | One-to-many |
| MedicationSchedule → DoseRecord | One-to-many |
| Patient → CareRelationship | One-to-many |
| User → Notification recipient | One-to-many |

## 6.4 Database Integrity and Migration

Flyway migration V1 creates the initial tables, keys and indexes. Later migrations add a medication active flag, support cancelled-dose state and restrict physical deletion of medications and schedules so dose history is not accidentally erased. Hibernate validates the schema on startup. The legacy database/schema.sql file is documented as a pointer and should not be treated as the authoritative initialization script.

### 6.5 Dose Record Integrity

Dose generation is performed under a per-patient transaction lock. Existing dose rows for the day are loaded together, and a uniqueness constraint on schedule, scheduled date and scheduled time acts as an additional idempotency guard. Dose-state transitions use conditional updates to help protect terminal states during concurrent requests.

### 6.6 Data Retention and Soft Deactivation

Medication deactivation is handled as a soft state change. Associated schedules are made inactive and pending doses are marked CANCELLED, while historical dose records are retained. Completed dose states are not rewritten. Cancelled doses are excluded from adherence analytics.

### 6.7 Privacy Considerations

- Password hashes and JWTs are not returned in monitoring responses.
- Patient-owned data routes derive the user from the authenticated principal.
- Care-team views require an active relationship of the matching role.
- Notification reads and updates are scoped to the recipient.
- Production credentials and secrets must be supplied through protected environment configuration.

# CHAPTER 7: IMPLEMENTATION AND FUNCTIONAL MODULES

## 7.1 Authentication Module

Registration accepts a full name, email and password. The server normalizes the email, checks uniqueness and hashes the password using BCrypt. Public registration creates a PATIENT account; clients cannot grant themselves a caregiver or doctor role by submitting a role field. Login verifies the submitted credentials and returns an access token and user summary.

## 7.2 Token Validation

The frontend stores the access token in sessionStorage through a dedicated token-storage utility. The API client attaches the token to protected requests. On the backend, the JWT authentication filter validates signature and expiry, loads the current account and checks that the token subject matches the stable database ID.

## 7.3 Medication Management

Medication endpoints allow authenticated patients to maintain their own medication records. Ownership checks ensure that a user cannot use another patient's medication identifier to view or modify the record. Deactivation preserves historical records and prevents inactive schedules from generating future pending doses.

## 7.4 Schedule Management

Schedules define when a medication is expected to be taken. The service layer creates dose records for relevant dates while preventing duplicate schedule/date/time combinations. Date calculations use a configured application clock and timezone to keep the calendar behaviour consistent.

## 7.5 Dose Tracking

Dose records capture scheduled occurrences and their status. State transitions are validated and applied conditionally. The system retains history for later review and analytics. The exact clinical meaning of a status should be interpreted as an application record, not an independent clinical assessment.

## 7.6 Adherence History and Analytics

The adherence module exposes date-filtered history and summary calculations. History supports pagination, with a default page size of 50 and a maximum of 100. Analytics accept a date range up to 366 days. Calculations use saved dose states; therefore, the quality of the output depends on the completeness and accuracy of recorded events.

### 7.7 Care-Team Relationship Module

A patient can invite an existing account by exact email lookup and specify a CAREGIVER or DOCTOR relationship. The invitee can accept or decline a pending request. The patient can revoke active access. Revoked relationships remain recorded, while a new invitation may be created later.

### 7.8 Caregiver Dashboard

The caregiver dashboard is designed for read-only monitoring of connected patients. It can show permitted patient summaries, saved doses for the current day and adherence analytics. Every monitoring request must have an active relationship of the matching role.

### 7.9 Doctor Dashboard

The doctor dashboard provides a similar read-only view for connected patients, including overviews, saved doses and analytics. It does not allow a connected doctor account to change a patient's dose status through monitoring routes.

### 7.10 Medication Interaction Checker

The patient submits medication identifiers to the interaction endpoint. The backend derives the patient identity from the token and verifies that every medication belongs to that patient before sending selected medication names to the configured provider. Mixed or foreign medication IDs fail the whole request.

**Important limitation:** the current default provider is a mock implementation and returns no clinical interaction data. A NO_DATA result does not prove that a combination is safe. The feature is informational and does not replace a doctor or pharmacist.

## 7.11 Notification Centre

SmartMed provides recipient-scoped in-app notifications for selected events such as missed-dose transitions, care-relationship changes and month-to-date adherence attention. The notification API supports pagination, unread-only filtering, unread count and read/unread state changes.

Duplicate notifications are prevented through a recipient/type/event key backed by a database uniqueness constraint. Care-team notifications are sent only to users with an active matching relationship at the relevant time. Adherence attention is an application rule: it requires at least three recorded doses and a result below the configured 70% product threshold. This is not a medically validated measure.

### 7.12 Frontend Component Organization

The frontend separates API modules, reusable components, pages, authentication utilities, date utilities and styling. Components include dashboards, monitoring widgets, notification controls, a care-team panel and an interaction checker. API functions centralize request behaviour and make it easier to test error handling.

### 7.13 Backend Package Organization

| Package | Responsibility |
|---|---|
| controller | REST endpoint mappings |
| service | Business rules and transactional operations |
| repository | Persistence access and bounded queries |
| entity | Database-mapped domain objects and enums |
| dto | Validated request and response contracts |
| security | JWT processing and authentication throttling |
| config | CORS, application clock, properties and Spring Security |
| exception | Consistent API error handling |

# CHAPTER 8: USER WORKFLOWS

## 8.1 Patient End-to-End Workflow

- Open the web application and register using name, email and password.
- Log in and receive an access token.
- Add medication details and configure a schedule.
- Review generated dose records and update dose status as appropriate.
- Open adherence history and analytics for the desired date range.
- Invite a caregiver or doctor when monitoring is needed.
- Review notifications and manage care-team consent over time.

## 8.2 Caregiver Workflow

- Receive a relationship invitation from a patient.
- Accept or decline the invitation.
- If accepted, view permitted connected-patient summaries and adherence information.
- Use the monitoring view as read-only; dose records cannot be changed from this role's monitoring routes.

## 8.3 Doctor Workflow

- Receive a doctor relationship invitation.
- Accept or decline the request.
- Review permitted patient overview, saved doses and analytics while the relationship remains active.
- Stop access when the patient revokes consent or the relationship becomes inactive.

## 8.4 Roles and Permissions

| Operation | Patient | Caregiver | Doctor |
|---|---|---|---|
| Manage own medications | Yes | No | No |
| Manage own schedules/dose records | Yes | No | No |
| View own adherence analytics | Yes | No | No |
| Invite/revoke care relationships | Patient controls consent | No | No |
| Read connected patient monitoring data | No (own data only) | Yes, active relationship | Yes, active relationship |
| Change patient dose status via monitoring | No | No | No |
| Manage own notifications | Yes | Recipient-scoped | Recipient-scoped |

## 8.5 Use-Case Narrative: Registration and Login

During registration, the frontend collects the required fields and submits them to the public registration endpoint. The backend validates the request, normalizes the email, checks for an existing account and persists a BCrypt hash. On login, the backend compares the submitted password against the stored hash and returns a JWT only when the credentials are valid. Generic invalid-credential messages avoid revealing whether a specific email exists.

### 8.6 Use-Case Narrative: Dose History

The patient selects a date range and opens adherence history. The client passes the selected range and pagination options to the API. The backend scopes the query to the authenticated patient, returns the requested page and supplies the data used by the interface to present saved dose events.

### 8.7 Use-Case Narrative: Consent Revocation

A patient may revoke an active care relationship. Subsequent monitoring requests must fail the active-relationship check, so a previously connected caregiver or doctor no longer has access through the monitored endpoints. The relationship remains in the record for traceability.

### 8.8 User Interface Principles

- Keep patient-owned and care-team views distinct.
- Show date ranges and status labels near analytics.
- Provide clear feedback for validation and API errors.
- Explain the limitations of interaction-check results.
- Make consent and revocation behaviour explicit.
- Avoid presenting application metrics as clinical advice.

# CHAPTER 9: TESTING AND RESULTS

## 9.1 Testing Objectives

Testing checks whether the system behaves according to its functional requirements and whether security boundaries are maintained. The repository includes backend integration tests, security-focused tests, migration tests and a frontend API-client test. This report describes the test structure present in the source; it does not claim a fresh successful test run unless separately executed.

## 9.2 Testing Types

| Test Type | Purpose |
|---|---|
| Unit tests | Verify focused utility or business behaviour |
| Integration tests | Exercise endpoints and persistence together |
| Authentication tests | Check registration, login and token rules |
| Authorization tests | Check role, ownership and relationship boundaries |
| Migration tests | Validate schema migration behaviour |
| Frontend API tests | Check request/response and client error handling |
| Manual acceptance tests | Verify user workflows in a browser |

## 9.3 Test Environment

The backend tests use the Spring Boot test framework and an in-memory H2 database profile. The project includes tests for authentication, medication operations, scheduling/adherence, care relationships, notifications, monitoring, analytics, interaction checks, rate limiting and Flyway migrations. Frontend tests are run through the npm test script.

## 9.4 Sample Test Cases

| ID | Test Scenario | Expected Result |
|---|---|---|
| TC-01 | Register with valid fields | Patient account created; token returned |
| TC-02 | Register duplicate email | Request rejected with validation/conflict response |
| TC-03 | Login with wrong password | 401 invalid credentials |
| TC-04 | Read protected profile without token | Request denied |
| TC-05 | Access another patient's medication | Request denied by ownership check |
| TC-06 | Caregiver reads without active relationship | Request denied |
| TC-07 | Patient revokes active relationship | Subsequent monitoring access denied |
| TC-08 | Submit foreign medication ID to interaction checker | Whole request rejected |
| TC-09 | Mark recipient notification read | Only recipient-owned record changes |
| TC-10 | Generate same scheduled dose twice | Duplicate dose prevented |

## 9.5 Security Test Considerations

- Ensure public registration cannot create elevated roles.
- Ensure a token subject matches the database user's stable identifier.
- Ensure password hashes and access tokens are excluded from user-facing responses.
- Ensure caregiver and doctor access requires a currently active matching relationship.
- Ensure queries are scoped to the authenticated user or authorized patient.
- Ensure notification operations are restricted to the recipient.
- Ensure rate limiting bounds repeated authentication failures and registration requests.

### 9.6 Acceptance Criteria

The main acceptance criteria are that authorized users can complete core medication workflows, saved records persist across sessions, date-filtered history is returned within configured bounds, care-team data is visible only with active consent, and API errors are returned consistently. The interaction checker must clearly communicate that mock/no-data results are not a clinical safety determination.

### 9.7 Results Summary

The supplied source code and test suite show implementation coverage for the core modules described in this report. A formal deployment acceptance should still include running backend and frontend builds, executing automated tests, checking database migrations, and testing the complete workflows in the target environment. No unsupported claim of production readiness is made here.

# CHAPTER 10: CONCLUSION AND FUTURE SCOPE

## 10.1 Conclusion

SmartMed demonstrates a layered web application for medication organization and adherence monitoring. It combines a typed React frontend with a Java/Spring Boot API and a MySQL database. Its functional scope includes authentication, medication records, scheduling, dose history, adherence analytics, consent-based care-team monitoring, an informational interaction-checking interface and recipient-owned notifications.

The project also demonstrates security-conscious design choices: BCrypt password hashing, JWT validation, database-backed role decisions, patient ownership checks, explicit consent relationships, bounded history/analytics queries and schema migrations. The retained dose history and soft-deactivation model support traceability.

SmartMed is an educational software project. Its current mock interaction provider does not supply real clinical interaction information, and its adherence rules are product logic rather than medically validated measurements. Users should consult qualified healthcare professionals for treatment decisions.

## 10.2 Future Enhancements

- Integrate a validated, licensed medication interaction data provider and clearly document coverage.
- Add configurable reminders and notification delivery channels with explicit user preferences.
- Add accessibility review and usability testing with representative users.
- Add audit logging and administrative tooling for trusted role provisioning.
- Improve deployment automation, monitoring, backups and recovery procedures.
- Evaluate multi-instance rate limiting with a shared gateway or distributed store.
- Add localization and carefully tested timezone preferences.
- Conduct a formal security review and privacy impact assessment.

## 10.3 Learning Outcomes

- Understanding client-server architecture and REST API design.
- Using Spring Security and JWT for stateless authentication.
- Applying ownership and role checks to sensitive operations.
- Designing relational data models and versioned migrations.
- Writing integration tests for business and security workflows.
- Separating presentation, business logic and persistence concerns.
- Recognizing the difference between software indicators and clinical advice.

### 10.4 Final Remarks

The project provides a foundation that can be extended through validated integrations, user testing and operational hardening. Future work should prioritize safety, privacy, data correctness and transparent communication of system limitations.

## REFERENCES


1. Spring Boot. (2025). Spring Boot Reference Documentation. https://docs.spring.io/spring-boot/
2. Spring Security. (2025). Spring Security Reference. https://docs.spring.io/spring-security/reference/
3. React. (2024). React Documentation. https://react.dev/
4. TypeScript. (2024). TypeScript Documentation. https://www.typescriptlang.org/docs/
5. Vite. (2024). Vite Guide. https://vite.dev/guide/
6. MySQL. (2025). MySQL 8.0 Reference Manual. https://dev.mysql.com/doc/
7. Flyway. (2025). Flyway Documentation. https://documentation.red-gate.com/flyway
8. JSON Web Token. (n.d.). Introduction to JSON Web Tokens. https://jwt.io/introduction
9. OWASP Foundation. (2021). OWASP Top 10 Web Application Security Risks. https://owasp.org/Top10/
10. Maven. (2025). Maven Documentation. https://maven.apache.org/guides/
11. Chart.js. (2025). Chart.js Documentation. https://www.chartjs.org/docs/
12. Rajasthan Technical University. (n.d.). Official Website. https://www.rtu.ac.in/
13. SmartMed project source code and documentation supplied for this report. README.md, docs/ARCHITECTURE.md, docs/AUTH.md and docs/PROJECT_STRUCTURE.md.

# APPENDIX A: LOCAL SETUP SUMMARY

## A.1 Prerequisites

- Install JDK 17 or later.
- Install Node.js 18+ and npm.
- Install Docker for the local MySQL container, or configure a compatible MySQL 8 instance.
- Use the included Maven wrapper for backend builds.

## A.2 Start Database

```
# From repository root
cp .env.example .env
cd database
docker compose --env-file ../.env up -d
```

## A.3 Start Backend

```
cd backend
# Set a fresh local JWT secret and local database variables.
./mvnw spring-boot:run
```

## A.4 Start Frontend

```
cd frontend
npm install
npm run dev
# Open http://localhost:5173
```

Use local-only credentials for development. Never commit .env files, production passwords, signing keys or truststore secrets. Configure the frontend API base URL and backend CORS allowlist consistently for the environment.

# APPENDIX B: IMPORTANT SAFETY AND SECURITY NOTES

## B.1 Clinical Limitations

SmartMed is not a medical device and is not intended to diagnose, prescribe, prevent or treat a disease. A missing interaction result is not proof that no interaction exists. Users must consult a doctor or pharmacist before making medication decisions.

## B.2 Secret Management

Generate a unique, cryptographically random JWT signing key for each environment. Store production secrets in the hosting provider's secret manager. Do not include usable secrets in source control, screenshots or project reports.

## B.3 Database and Transport Security

Use explicit database TLS configuration. Disabled database TLS is only described for local development or a specifically documented encrypted private-network deployment. Public/external database connections should use certificate and hostname verification.

## B.4 Consent and Privacy

Care-team access should be granted only with patient consent and removed when consent is revoked. Application operators should define retention, access review, backup and incident-response procedures before real-world deployment.

## B.5 Report Integrity

This report is based on the submitted source archive and its documentation. It describes the intended architecture and visible implementation structure. Actual runtime behaviour, deployment availability and test outcomes should be confirmed by executing the project in its target environment.